COESIONE ITALIA 21-27 PIEMONTE | Cofinanziato dall'Unione europea | REGIONE PIEMONTE

Cloud Specialist [D68444-15-2025-23]

Unità Formativa : Programmazione Python

Docente: Massimo PAPA

Titolo argomento: Soluzione primi esercizi sulle funzioni

Intervento realizzato da ITS TECNOLOGIE DELL'INFORMAZIONE E DELLA COMUNICAZIONE

Primi esercizi sulle funzioni

Indicazioni sulla consegna **3**
Struttura del codice sorgente 4
Testo Esercizi **6**
Primo Esercizio 6
Secondo Esercizio 6
Terzo Esercizio 6
Quarto Esercizio 6
Soluzione Esercizi **8**
Primo Esercizio 8
Secondo Esercizio 11
Terzo Esercizio 14
Quarto Esercizio 15

Indicazioni sulla consegna

Implementare gli algoritmi risolutivi dei seguenti esercizi codificandoli in linguaggio python. Partire analizzando il problema seguendo la seguente traccia:

- Quali sono gli input del problema?
- Quali sono gli output?
- Suddividi il problema in problemi più semplici e se lo ritieni opportuno descrivi la soluzione di ogni sottoproblema con un flow-chart
- Andrai a rappresentare ogni algoritmo che risolve un sottoproblema come una funzione
- La funzione restituisce un valore? Se si di che tipo?
- Ogni funzione accetta una lista di parametri formali? Se si di che tipo?
- Esegui la codifica partendo dal programma principale, al suo interno vai a richiamare le funzioni che poi andrai successivamente a definire.
- Continua la codifica dichiarando e definendo tutte le funzioni prima del programma principale che richiama tutte le altre. In testa alle funzioni scrivi un commento che descrive la funzione stessa , cosa restituisce e quali sono i parametri formali.
- Scrivi la sezione dell'inizializzazione variabili scrivendo in un commento il significa della variabile stessa
- Verifica la codifica utilizzando input di test, cercando di provare anche i casi limite.

Attenzione: non utilizzare strutture complesse in nessuno dei seguenti esercizi.

Struttura del codice sorgente

Relativamente alle indicazioni di scrittura del codice, utilizza il seguente schema generale:

```
'''
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Testo esercizio

'''

##
## Funzioni:
##


def fun(param1: int, param2: float) -> int:
  '''
     Funzione: fun
      Template per costruire le funzioni
      Parametri formali:
          int Param1 -> descrizione Param1
          float Param2 -> descrizione Param2
      Valore di ritorno:
          int -> descrizione valore di ritorno
  '''
  retValue = 5 # Valore di ritorno della funzione

  return retValue


'''
    Programma principale
    Descrizione sintetica funzionalità
    del programma principale.
'''

# Sezione di input Dati

# Inizializzazioni variabili

# Elaborazione
```

```
# Eventuali sotto processi di Elaborazione

# Sezione di output
```

Testo Esercizi

Primo Esercizio

Creare una funzione che riceva una quantità di tempo in formato ore, minuti e secondi e la restituisca espressa solamente in secondi. Successivamente creare un programma principale che chieda in input due quantità di tempo e stampi in output quale quantità di tempo è maggiore. La funzione deve avere i parametri formali con valori predefiniti.

Secondo Esercizio

Creare una funzione che abbia come parametri formali un numero arbitrario di valori numerici. Si vuole che restituisca la somma dei soli numeri pari e il prodotto dei soli numeri dispari. Successivamente creare un programma che richiami tale funzione e che stampi in output i risultati . No standard input.

Terzo Esercizio

Conversione temperatura: implementare una funzione convertiCF che converta una temperatura espressa in gradi Fahrenheit in una temperatura espressa in gradi Celsius. Usare la seguente formula:

$$F = (9 / 5) * C + 32$$

Creare un programma principale che richiami la funzione e ne stampi il risultato visualizzando solo 3 cifre decimali.

Quarto Esercizio

La seguente è la formula per valutare numericamente il numero di Nepero e:

(1) $$e := \sum_{n=0}^{+\infty} \frac{1}{n!} = \frac{1}{0!} + \frac{1}{1!} + \ldots + \frac{1}{n!} + \ldots$$

4.a Scrivere una funzione che restituisca un valore approssimato di e all'ennesimo termine, dove N è inserito come parametro formale alla funzione.

Esempio: valuta_e(3) restituirà il valore di e calcolato con tre termini della (1), cioè:

$$e= 1/0! + 1/1! + 1/2! = 1 + 1 + 0.5 = 2.5$$

La funzione calcola_e deve richiamare la funzione di calcolo del fattoriale.

Scrivere il codice della funzione e il programma principale che la chiama chiedendo in input il numero N.

4.b Supponendo di porre il numero di `Nepero = 2.718281828459045` dico che

errore = valuta_e(N) - Nepero

sia l'errore che commetto nella valutazione di e.

Modificare la funzione che restituisce la valutazione di e con N termini andando a far restituire anche l'errore commesso nella valutazione. Suggerimento: la funzione valuta_e(N) dovrà restituire due valori, `2.718281828459045` potrebbe essere memorizzato in una variabile globale.

esempio: valuta_e(3) restituisce il valore calcolato nel punto 4.a 2.5 e 0,218281828459045 che rappresenta la differenza tra 2.718281828459045 e 2.5

Soluzione Esercizi

Primo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Esercizio: Primo Esercizio
    Creare una funzione che riceva una quantità di tempo
    in formato ore, minuti e secondi e la restituisca
    espressa solamente in secondi. Successivamente creare
    un programma principale che chieda in input due
    quantità di tempo e stampi in output quale quantità di
    tempo è maggiore. La funzione deve avere i parametri
    formali con valori predefiniti.

"""

##
## Funzioni:
##


def inserisciTempo() -> tuple:
    """
    Funzione: inserisciTempo
    Permette di inserire una quantità di tempo
    espressa in ore, minuti, secondi
    Parametri formali:
       int hh -> ore (0-23)
       int mm -> minuti (0-59)
       int ss -> minuti (0-59)
     Valore di ritorno:
         int secondi -> quantità di tempo espressa in secondi
    """
    # inizializzazione variabili
    hh = -1  # ore fuori dominio
    mm = -1  # ore fuori dominio
    ss = -1  # ore fuori dominio
```

```
    # Inserimento ore
    while hh < 0 or hh > 23:
        hh = int(input("Inserisci hh [0-23]:"))

    # Inserimento minuti
    while mm < 0 or mm > 59:
        mm = int(input("Inserisci mm [0-59]:"))

    # Inserimento secondi
    while ss < 0 or ss > 59:
        ss = int(input("Inserisci ss [0-59]:"))

    return hh, mm, ss


def conversioneSecondi(hh: int = 0, mm: int = 0, ss: int = 0) -> int:
    """
    Funzione: conversioneSecondi
    Converte una quantità di tempo
    espressa in ore, minuti, secondi
    in secondi
    Parametri formali:
       int hh -> ore (0-23)
       int mm -> minuti (0-59)
       int ss -> minuti (0-59)
     Valore di ritorno:
         int secondi -> quantità di tempo espressa in secondi
    """
    return ss + 60 * mm + 3600 * hh


"""
    Programma principale
    Si inseriscono due quantità di tempo
    e si stampa quale delle due è maggiore
"""

# Sezione di input Dati
print("Inserisci tempo1")
hh1, mm1, ss1 = inserisciTempo()
print("Inserisci tempo2")
hh2, mm2, ss2 = inserisciTempo()

# Conversione, confronto e stampa
if conversioneSecondi(hh1, mm1, ss1) > conversioneSecondi(hh2, mm2,
ss2):
```

```
    print("Maggiore tempo1")
else:
    print("Maggiore o uguale tempo2")
```

Secondo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Secondo Esercizio
    Creare una funzione che abbia come parametri
    formali un numero arbitrario di valori numerici.
    Si vuole che restituisca la somma dei soli numeri
    pari e il prodotto dei soli numeri dispari.
    Successivamente creare un programma che richiami
    tale funzione e che stampi in output i risultati.
    No standard input.

"""


def sommaProdotto(*numeri) -> tuple:
    """
    Funzione: sommaProdotto
    Esegue la somma sei numeri pari e
    il prodotto dei numeri dispari
    passati come parametri formali.
    Se esistono num pari e dispari allora
    errorCode = 0,
    altrimenti
    se non esistono num pari allora
    errorCode = 1
    se non esistono num dispari allora
    errorCode = 2
    se non si passa alcun numero allora
    errorCode = 3

    Parametri formali:
        sequenza di numeri interi
    Valore di ritorno:
        int,int,int -> Somma numeri pari, prodotto numeri dispari,
                       errorCode (0 -> OK, 1-> noPARI,
                                  2-> noDispari, 3->NONUM)
    """
    # inizializzazione variabili
    sommaPari = 0  # accumulatore numeri pari
```

```
    prodottoDispari = 1  # accumulatore numeri dispari
    almenoUnPari = False  # flag esistenza pari
    almenoUnDispari = False  # flag esistenza dispari

    # loop principale di calcolo
    for num in numeri:
        if num % 2 == 0:
            sommaPari += num
            almenoUnPari = True
        else:
            prodottoDispari *= num
            almenoUnDispari = True

    # setto errorCode
    if almenoUnPari and almenoUnDispari:
        errorCode = 0
    elif almenoUnPari and not almenoUnDispari:
        errorCode = 2
    elif not almenoUnPari and almenoUnDispari:
        errorCode = 1
    else:
        errorCode = 3
    return sommaPari, prodottoDispari, errorCode


def stampaRisultato(sommaPari, prodottoDispari, errorCode) -> None:
    """
    Funzione: stampaRisultato
    Interpreta e stampa il risultato
    dei valori di ritorno della
    Funzione sommaProdotto()

    Parametri formali:
        int sommaPari -> somma val pari sequenza
        int prodottoDispari -> prodotto val dispari sequenza
        int errorCode -> codice errore della funzione sommaProdotto
    Valore di ritorno:
        None
    """
    # interpretazione codice di errore
    if errorCode == 0:
        print(
            "Somma dei num pari:",
            sommaPari,
            "prodotto dei num dispari:",
```

```
            prodottoDispari,
        )
    elif errorCode == 1:
        print("PARI NON PRESENTI, Prodotto dei dispari:",
prodottoDispari)
    elif errorCode == 2:
        print("DISPARI NON PRESENTI, somma dei dispari:", sommaPari)
    else:
        print("NON SONO PRESENTI NUMERI!")


"""
Programma principale
Prova della funzione sommaProdotto
richiamandola almeno due volte

"""
sommaP, prodottoD, errorC = sommaProdotto(1, 2, 3, 4)
stampaRisultato(sommaP, prodottoD, errorC)

sommaP, prodottoD, errorC = sommaProdotto(1, 3)
stampaRisultato(sommaP, prodottoD, errorC)

sommaP, prodottoD, errorC = sommaProdotto(2, 4)
stampaRisultato(sommaP, prodottoD, errorC)

sommaP, prodottoD, errorC = sommaProdotto()
stampaRisultato(sommaP, prodottoD, errorC)
```

Terzo Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Conversione temperatura: implementare una funzione
    convertiCF che la converta in gradi Fahrenheit di una temperatura
    in gradi Celsius. Usare la seguente formula:
            F = ( 9 / 5 ) * C + 32
    Creare un programma principale che richiami la funzione e ne stampi
    il risultato visualizzando solo 3 cifre decimali.

"""


def convertiCF(gradiC:float) -> float:
    """
    Funzione: convertiCF
    Converte argomento espresso in gradi Celsius
    in gradi Fahrenheit usando la formula:
        F = ( 9 / 5 ) * C + 32

    Parametri formali:
        float gradiC -> temperatura da convertire
    Valore di ritorno:
        float gradiF -> temperatura convertita
    """
    gradiF = 9 / 5 * gradiC +32
    return gradiF

"""
Programma principale
Prova della funzione convertiCF

"""
gradi=convertiCF(27.4654)
# Stampo utilizzando solo 3 cifre decimali
print(gradi)
print(f"{gradi:.3f}")
```

Quarto Esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Quarto Esercizio
    4.a Scrivere una funzione che restituisca un valore approssimato
    di e all'ennesimo termine, dove N è inserito come parametro
    formale alla funzione.
    Esempio: valuta_e(3) restituirà il valore di e calcolato con tre
    termini della (1), cioè:
        e= 1/0! + 1/1! + 1/2! = 1 + 1 + 0.5 = 2.5
    La funzione calcola_e deve richiamare la funzione di calcolo del
    fattoriale.
    Scrivere il codice della funzione è il programma principale che la
    chiama chiedendo in input il numero N.

    4.b Supponendo di porre nepero = 2.718281828459045 dico che
        errore = valuta_e(N) - nepero
    sia l'errore che commetto nella valutazione di e.
    Modificare la funzione che restituisce la valutazione di e con N
    termini andando a far restituire anche l'errore commesso nella
    valutazione.
    Suggerimento: la funzione valuta_e(N) dovrà restituire due valori,
       2.718281828459045 potrebbe essere memorizzato in una variabile
globale.
    esempio: valuta_e(3) restituisce il valore calcolato nel punto 4.a
    2.5 e 0,218281828459045 che rappresenta la differenza tra
    2.718281828459045 e 2.5

"""

# Variabile globale che contiene il numero e considerato valore vero
NumeroE = 2.718281828459045


def fatt(n: int) -> int:
    """
    Funzione fatt
    Calcola il fattoriale del parametro
    della funzione.
```

```
    Parametri formali:
        int n -> valore su cui calcolare il fattoriale
    Valore di ritorno:
        int fatt -> fattoriale di n
    """
    # inizializzo la variabile che conterrà il fattoriale di n
    fatt = 1
    # loop di calcolo del fattoriale
    for i in range(2, n + 1):
        fatt *= i
    return fatt


def Nepero(N: int = 5) -> float:
    """
    Funzione Nepero
    Calcola il numero di nepero
    approssimato all'ennesimo termine.

    Parametri formali:
        int N -> indica il termine a cui troncare lo sviluppo
    Valore di ritorno:
        int nepero -> numero di Nepero approssimato
    """

    # inizializzazione variabile che conterrà numero di nepero
    nepero = 1
    # loop di calcolo della serie
    for n in range(1, N):
        nepero += 1 / fatt(n)
    return nepero


def NeperoErr(N: int = 5) -> tuple:
    """
    Funzione NeperoErr
    Calcola il numero di nepero
    approssimato all'ennesimo termine.

    Parametri formali:
        int N -> indica il termine a cui troncare lo sviluppo
    Valore di ritorno:
        int nepero -> numero di Nepero approssimato
        int err -> errore commesso nell'approssimazione
    """
```

```
    # inizializzazione variabile che conterrà numero di nepero
    nepero = 1
    # loop di calcolo della serie
    for n in range(1, N):
        nepero += 1 / fatt(n)
    # Valutazione dell'errore
    err = abs(nepero - NumeroE)

    return nepero, err

'''
Programma principale
Parte 4.a
'''
print(Nepero(3))

'''
Programma principale
Parte 4.b
'''
print(NeperoErr(3))

# prova con approssimazioni sempre maggiori
for i in range(1, 20):
    print(NeperoErr(i))
```